# AdsKG-Agent: Knowledge-Graph RAG for Table Selection in a Retail-Media Warehouse

Capstone design doc · AI Engineering Cohort · Anshika Chaurasia (solo) · October 2026

## 1. Problem Statement

Ad analysts ask questions such as *"CTR by brand for female users in age level 3 yesterday"* or *"unique reach of campaign 83237 in the last 6 hours"*. A retail-media warehouse holds many overlapping tables — the same metric at hourly/daily/weekly grain, raw vs. invalid-traffic-filtered, impressions vs. unique reach, real-time partial vs. reconciled. LLM text-to-SQL fails here because the full schema does not fit the prompt and table names do not explain which variant fits the question. Wrong-table answers *look* correct, so errors go unnoticed.

**Task:** given one natural-language analytics question (plus prior conversation turns), select the minimal correct set of tables and join path, generate DuckDB SQL, execute it, and return the answer with caveats. **In scope:** read-only questions over ~28 ad-performance tables; multi-turn follow-ups. **Out of scope:** root-cause ("why") analysis, recommendations, write operations, dashboards.

**Motivation:** my company's ads analytics agent currently answers "what" questions; this project prototypes the vector-DB, GraphRAG and memory layer it lacks, built fresh on public data (no company code or data).

## 2. Data & Ingestion Pipeline

| Source (public) | Contents (verified on load) |
|---|---|
| Taobao Ad Display/Click (Alibaba, via Kaggle) | 26.56M impressions, 1.37M clicks (CTR 5.14%), 1.14M users, May 6–13 2017; 847k ads → 423k campaigns, 256k advertisers, 99.8k brands, 6.8k categories; 1.06M user profiles (gender, age, spend level, city tier) |

- **Load & clean** (DuckDB): fix reserved-word column (`user`), parse literal "NULL" brands (29% of ads), flag 1.5M impressions with no user profile (LEFT JOIN required).
- **Model ~28 tables** with documented SQL: staging, 7 dimensions, event facts, and variant families — grain (hourly/daily/weekly), traffic quality (raw vs. valid), reach vs. impressions, simulated real-time slice (last N hours, disclosed). Click-value = clicks × item price, labelled as a proxy (dataset has no spend/revenue).
- **Catalog extraction:** per table a *table card* (description, grain, freshness, columns, sample values, join keys) → embedded into Qdrant; tables/columns/metrics/business terms and edges → Neo4j knowledge graph.
- **Pivot already made:** the purchase `behavior_log` (needed for attribution variants) is absent from the Kaggle mirror and Tianchi download failed; attribution variants replaced by traffic-quality and reach variants.

## 3. Architecture

**Flow:** question → **query rewrite** (extract metric, entities, grain, time window) → **hybrid retrieval** in Qdrant (BM25 sparse + dense, metadata filters) → **cross-encoder rerank** → seed tables → **graph expansion** in Neo4j (shortest join paths between seeds, VARIANT_OF siblings, metric → column mapping) → **LangGraph ReAct agent** selects tables, writes SQL, executes, repairs on error (max 3) → answer + caveats.

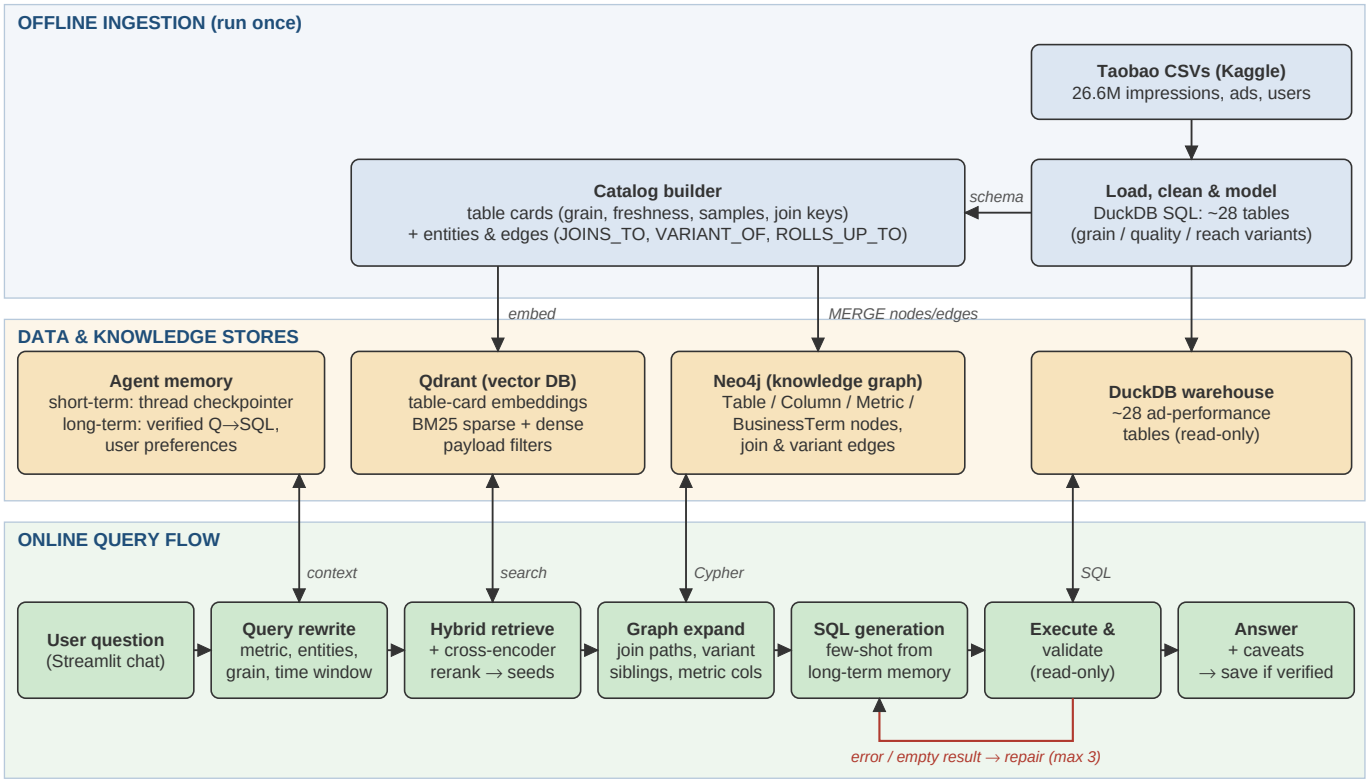


*Figure 1. System architecture: offline catalog ingestion feeds the vector store and knowledge graph; the online LangGraph agent reads memory, Qdrant, Neo4j and DuckDB at each step.*

| Component | Design |
|---|---|
| Knowledge graph (Neo4j) | (:BusinessTerm)-[:MEANS]->(:Metric)-[:COMPUTED_FROM]->(:Column); (:Table)-[:HAS_COLUMN]->(:Column)-[:JOINS_TO]->(:Column); (:Table)-[:VARIANT_OF {differs_by}]->(:Table); [:ROLLS_UP_TO] grain edges; dimension hierarchy advertiser→campaign→adgroup; (:VerifiedQuery)-[:USED]->(:Table) |
| Agent tools | search_catalog, get_join_path (Cypher shortestPath), list_variants, describe_table, sample_rows, run_sql (read-only) |
| Short-term memory | LangGraph checkpointer per conversation thread: resolves follow-ups ("now split that by age") |
| Long-term memory | Verified question→SQL pairs in Qdrant (retrieved as few-shot examples, linked into the KG); user preferences (e.g. default traffic filter) in the LangGraph store |

## 4. Approaches Compared (simple baseline first, one technique added per step)

| ID | Approach | Isolates the value of |
|---|---|---|
| A | Full schema in prompt, one-shot SQL | Baseline |
| B | Vanilla vector RAG: dense top-k table cards | Vector DB retrieval |
| C | Advanced RAG: B + hybrid BM25 + rerank + query rewrite | Advanced retrieval |
| D | GraphRAG: C + Neo4j join-path and variant expansion | Knowledge graph |
| E | LangGraph agent using D as tools + short/long-term memory | Agentic loop & memory |

Final choice is justified by accuracy per unit cost/latency; if D matches E at lower cost, D is chosen.

## 5. Evaluation Criteria

- **Benchmark (hand-built, executed):** ~100 questions in 3 tiers — single-table, multi-hop join, variant/ambiguity (grain, freshness, traffic quality) — each with gold tables and gold SQL whose result is executed and checked; plus 20 multi-turn conversations (3–4 turns) for memory. Split dev/test; test is untouched until final runs.
- **Quantitative:** table recall@k and set-F1; execution accuracy (result-set match vs. gold); join-path correctness; repair rate; latency, tokens and $ per question; memory: follow-up accuracy and accuracy on paraphrased repeats, memory on vs. off.
- **LLM-as-judge** (rubric, 1–5): answer addresses the question, caveats correct (e.g. partial real-time data), faithful to SQL result. Judge calibrated on 30 human-graded answers (report agreement).
- **Qualitative:** failure taxonomy (wrong variant, missing bridge table, wrong join, over-expansion, hallucinated column) documented in the README failure-analysis section.
- **Hypotheses:** D > C > B on multi-hop and variant tiers; roughly equal on single-table; E adds most on multi-turn. External sanity check (stretch): BIRD dev subset.

## 6. Framework Justification

| Choice | Why | Alternative considered |
|---|---|---|
| DuckDB | Columnar, in-process; 26M rows aggregate in seconds on a laptop; no server | Postgres (needs server) |
| Qdrant | Native hybrid sparse+dense search with payload filters; runs embedded or in Docker; production-grade | Chroma (no native hybrid), pgvector |
| Neo4j | Typed nodes/edges + Cypher path queries match the catalog structure; industry-standard graph DB to transfer to work | NetworkX (no query language/persistence) |
| LangGraph | Explicit state graph for retrieve→SQL→validate→repair loop; built-in checkpointer (short-term) and store (long-term memory) | Pydantic AI, smolagents (weaker memory/state) |
| OpenAI models + text-embedding-3-small; bge-reranker | Course-provided key; open-source cross-encoder for reranking | — |
| Streamlit (optional UI) | Chat UI showing retrieved tables, graph path and SQL | Gradio |

## 7. Risks & Mitigations

Learning three new tools in a short build → must-have = A, B, D, E; C's rerank and long-term memory are should-haves. Benchmark bias from self-designed tables → tiers reported separately, variants derived from standard warehouse patterns, optional BIRD check. Anonymised IDs make questions less natural → dimension labels generated deterministically and disclosed.